Tribhuvan University

Faculty of Humanities & Social Sciences

OFFICE OF THE DEAN

Network Programming

BCA — Semester 6 — 2024 — Model Answer Sheet

Subject Code: CACS355

Note: This is a model answer sheet for study purposes. Answers are based on standard curriculum topics.

Group B

Attempt any SIX questions. [6×5=30]

2. Define network programming. Explain the client-server model with a diagram.

Answer: Network Programming: Writing programs that communicate over a network using protocols like TCP/IP, UDP. Enables distributed applications, resource sharing, and remote communication. **Client-Server Model: Server:** Passive entity that waits for requests, provides services. **Client:** Active entity that requests services. **Process:** 1) Server starts and listens on a port. 2) Client initiates connection to server's IP and port. 3) Server accepts connection. 4) Data exchange occurs. 5) Connection closes. **Diagram:** Client → Request → Server → Response → Client. **Types:** Iterative (handles one client at a time), Concurrent (uses threads/processes for multiple clients).

3. Differentiate between TCP and UDP protocols. When would you use each?

Answer: TCP (Transmission Control Protocol): Connection-oriented, reliable, ordered delivery, error checking, flow control, congestion control. Uses three-way handshake. Higher overhead. **Used for:** Web browsing (HTTP), email (SMTP), file transfer (FTP). **UDP (User Datagram Protocol):** Connectionless, unreliable, no ordering, no error recovery, low overhead, faster. **Used for:** Streaming video/audio, online gaming, DNS queries, VoIP. **Comparison:** TCP is like certified mail (guaranteed delivery); UDP is like regular mail (fast but no guarantee).

4. Explain the working of Socket and ServerSocket classes in Java with example.

Answer: Socket: Endpoint for communication between two machines. **ServerSocket:** Waits for client requests on a specific port. **Example: Server:** ServerSocket server = new ServerSocket(8080); Socket client = server.accept(); BufferedReader in = new BufferedReader(new InputStreamReader(client.getInputStream())); PrintWriter out = new PrintWriter(client.getOutputStream()); Read request and send response. **Client:** Socket socket = new Socket('localhost', 8080); PrintWriter out = new PrintWriter(socket.getOutputStream()); out.println('Hello'); BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream())); Read response. **Key Methods:** accept(), getInputStream(), getOutputStream(), close().

5. What is URL? Explain the components of URL and how to process URLs in Java.

Answer: URL (Uniform Resource Locator): Address of a resource on the Internet. **Components: Protocol:** http, https, ftp. **Host:** Domain name or IP address. **Port:** Optional (default 80 for HTTP, 443 for HTTPS). **Path:**

Resource location on server. **Query:** Parameters after '?'. **Fragment:** Section identifier after '#'. **Example:** https://example.com:8080/path/page.html?id=1#section. **Java URL Class:** URL url = new URL('https://example.com'); URLConnection conn = url.openConnection(); BufferedReader reader = new BufferedReader(new InputStreamReader(conn.getInputStream())); Read content line by line. **HttpURLConnection:** For HTTP-specific features (GET/POST, headers, response codes).

6. Describe datagram communication. How does DatagramSocket and DatagramPacket work?

Answer: Datagram Communication: Connectionless packet-based communication using UDP. Each packet (datagram) is independent and may arrive out of order or be lost. **DatagramSocket:** Socket for sending/receiving datagrams. Can be bound to a port. **DatagramPacket:** Contains data, length, destination address, and port. **Example: Sender:** DatagramSocket socket = new DatagramSocket(); byte[] buf = msg.getBytes(); DatagramPacket packet = new DatagramPacket(buf, buf.length, InetAddress.getByName('host'), 4445); socket.send(packet); **Receiver:** DatagramSocket socket = new DatagramSocket(4445); byte[] buf = new byte[256]; DatagramPacket packet = new DatagramPacket(buf, buf.length); socket.receive(packet); String received = new String(packet.getData(), 0, packet.getLength()); **Use Cases:** Broadcasting, multiplayer games, streaming.

7. Explain the concept of multithreading in network programming. Why is it important?

Answer: Multithreading in Network Programming: Allows a server to handle multiple clients simultaneously by creating a new thread for each connection. **Importance:** 1) **Concurrency:** Multiple clients served at once. 2) **Responsiveness:** No client waits for others to finish. 3) **Resource Utilization:** CPU works while one thread waits for I/O. 4) **Scalability:** Server can handle growing client base. **Implementation:** ServerSocket server = new ServerSocket(8080); while (true) { Socket client = server.accept(); new Thread(new ClientHandler(client)).start(); }. **Alternatives:** Thread pools (limit threads), NIO (non-blocking I/O with selectors), async programming. **Challenges:** Race conditions, deadlocks, thread overhead.

8. What is network security? Explain SSL/TLS and how it secures network communication.

Answer: Network Security: Protection of data during transmission over networks from unauthorized access, modification, or destruction. **SSL/TLS (Secure Sockets Layer/Transport Layer Security):** Cryptographic protocols providing secure communication. **How it works:** 1) **Handshake:** Client and server agree on encryption algorithm and exchange keys. 2) **Authentication:** Server presents certificate (verified by client). 3) **Key Exchange:** Symmetric session key generated using asymmetric encryption. 4) **Encrypted Communication:** All data encrypted with session key. **Features:** Confidentiality (encryption), Integrity (MAC), Authentication (certificates). **Java Implementation:** Use SSLSocket and SSLServerSocket instead of regular Socket. **HTTPS:** HTTP over TLS.

Group C

Attempt any TWO questions. [2×10=20]

9. Write a Java program to create a simple chat application using TCP sockets. Explain how multiple clients can be handled using threads.

Answer: Chat Application: Server: Creates ServerSocket on a port. In a loop, accepts client connections and starts a new Thread for each. Each thread reads messages from its client and broadcasts to all connected clients. Uses a shared Set of PrintWriters for broadcasting. **Client:** Connects to server using Socket. Starts a ReaderThread to listen for incoming messages. Main thread reads user input and sends to server. **Multi-client Handling:** Server maintains List of active clients. When message received, iterate through list and send to all except sender. **Synchronization:** Access to shared client list must be synchronized. **Improvements:** Use thread pool instead of unlimited threads, add username support, private messaging.

10. Explain the OSI model and TCP/IP model. Compare their layers and functions in detail.

Answer: OSI Model (7 layers): 1) **Physical:** Bits on wire, cables, signals. 2) **Data Link:** Frames, MAC addresses, error detection. 3) **Network:** Packets, IP addresses, routing. 4) **Transport:** Segments, TCP/UDP, end-to-end delivery. 5) **Session:** Session management, dialog control. 6) **Presentation:** Data formatting, encryption, compression. 7) **Application:** HTTP, FTP, SMTP. **TCP/IP Model (4 layers):** 1) **Network Access:** Physical + Data Link. 2) **Internet:** Network layer (IP). 3) **Transport:** TCP/UDP. 4) **Application:** Session + Presentation + Application. **Comparison:** OSI is theoretical, TCP/IP is practical. OSI has 7 layers, TCP/IP has 4. TCP/IP came first; OSI was developed later as reference model. TCP/IP is the foundation of the Internet.

11. What is an HTTP protocol? Write a Java program to send an HTTP GET request and process the response using URLConnection or HttpClient.

Answer: HTTP (Hypertext Transfer Protocol): Application-layer protocol for web communication. **Methods:** GET (retrieve), POST (submit), PUT (update), DELETE (remove), HEAD, OPTIONS. **Status Codes:** 200 OK, 404 Not Found, 500 Server Error. **Java URLConnection:** URL url = new URL('https://api.example.com/data'); HttpURLConnection conn = (HttpURLConnection) url.openConnection(); conn.setRequestMethod('GET'); conn.setRequestProperty('Accept', 'application/json'); BufferedReader reader = new BufferedReader(new InputStreamReader(conn.getInputStream())); Read response. **Java HttpClient (Java 11+):** HttpClient client = HttpClient.newHttpClient(); HttpRequest request = HttpRequest.newBuilder().uri(URI.create('https://api.example.com')).GET().build(); HttpResponse response = client.send(request, HttpResponse.BodyHandlers.ofString()); System.out.println(response.body()); **Advantages of HttpClient:** Async support, HTTP/2, cleaner API.